Light

Data Representation — Numbers

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Number systems
2. Binary arithmetic
3. Hexadecimal
Worked examples
Quick reference
Typical pitfalls

OCR GCSE Computer Science — **Data Representation: Numbers (J277/01 Section 1.2.3—1.2.4)**. This topic covers how computers represent numbers using binary (base 2) and hexadecimal (base 16), including conversions between number systems, binary addition, binary shifts, and understanding number ranges.

Exam questions typically ask you to **convert** between binary, denary and hexadecimal, **perform** binary addition and shifts, and **calculate** the range of values for a given number of bits. The 2024 exam included an 8-bit binary to denary conversion table, binary addition with working, and questions about maximum values and colour depth.

What you must know

This checklist covers the key exam-visible concepts. You should be able to perform each skill quickly and accurately.

- **Convert** between 8-bit binary and denary (both directions).
- **Convert** between denary and 2-digit hexadecimal (both directions).
- **Convert** between binary and hexadecimal (both directions).
- **Add** two 8-bit binary numbers, showing working and handling carries.
- **Perform** left and right binary shifts and explain their effect on the value.
- **Calculate** the range of values (minimum and maximum) for a given number of bits.

- **Calculate** the number of bits needed to represent a given number of values (e.g. characters in a character set, colours).
- **Explain** why computers use binary to represent data.

> **Specification note.** OCR J277 Section 1.2.4 covers units, binary, denary, hexadecimal, binary addition (with overflow), and binary shifts. Questions typically use 8-bit binary numbers. The September 2024 update clarified the wording around character representation.

Toolkit

Why computers use binary

Computers use **binary (base 2)** because electronic circuits have two states: **on** (1) and **off** (0). This makes binary reliable, simple to implement in hardware, and resistant to electrical noise. All data — numbers, text, images, sound — must be converted to binary for a computer to process it.

Place values – 8-bit binary

Each column in binary represents a power of 2, doubling from right to left:

128	64	32	16	8	4	2	1
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

To convert binary to denary: add the place values where there is a 1.
To convert denary to binary: subtract the largest place value that fits, put a 1 there, repeat.

Hexadecimal digits

Denary	0	1	2	3	4	5	6	7	8
Hex	0	1	2	3	4	5	6	7	8
Binary	0000	0001	0010	0011	0100	0101	0110	0111	1000

Key insight: Each hex digit represents exactly 4 binary bits (a nibble). This is why hex is useful — it's a compact way to write binary.

Number ranges

Number of bits (n)	Minimum value	Maximum value	Number of different values
4 bits	0	15	16 (2^4)

Number of bits (n)	Minimum value	Maximum value	Number of different values
6 bits	0	63	64 (2^6)
7 bits	0	127	128 (2^7)
8 bits	0	255	256 (2^8)
n bits	0	$2^n - 1$	2^n

Formula: With n bits, you can represent 2^n different values (0 to $2^n - 1$).

Binary shift rules

Left shift by n places: Multiplies the value by 2^n. Bits move left, zeros fill from the right. Bits that go off the left edge are lost.

Right shift by n places: Divides the value by 2^n (integer division, remainder lost). Bits move right, zeros fill from the left. Bits that go off the right edge are lost.

Shift	Effect	Example: 00001100 (12)
Left shift 1	× 2	00011000 (24)
Left shift 2	× 4	00110000 (48)
Right shift 1	÷ 2	00000110 (6)
Right shift 2	÷ 4	00000011 (3)

Data units

Data is measured in standard units. Each unit is 1,000 times larger than the previous (using decimal prefixes):

Unit	Symbol	Size	Example use
Bit	b	1 binary digit (0 or 1)	Smallest unit of data
Nibble	—	4 bits	One hex digit
Byte	B	8 bits	One character (ASCII)
Kilobyte	KB	1,000 bytes	Short text document
Megabyte	MB	1,000 KB	MP3 song, photo
Gigabyte	GB	1,000 MB	HD film, game

Unit	Symbol	Size	Example use
Terabyte	TB	1,000 GB	Hard drive capacity
Petabyte	PB	1,000 TB	Data centre storage

Converting units: To convert to a smaller unit, multiply by 1,000. To convert to a larger unit, divide by 1,000.
Example: 2.5 GB = 2.5 × 1,000 = 2,500 MB. Or: 500,000 KB = 500,000 ÷ 1,000 = 500 MB.

Binary addition rules

- 0 + 0 = 0
- 0 + 1 = 1
- 1 + 0 = 1
- 1 + 1 = 10 (0, carry 1)
- 1 + 1 + 1 = 11 (1, carry 1)

Overflow: If the result needs more bits than available (e.g. 9 bits for an 8-bit system), this is called **overflow**. The extra bit is lost, giving an incorrect result.

Exam tip: Always show your working for binary addition. Write the carry digits above the columns. Even if your final answer is wrong, you can gain marks for correct method.

1. Number systems and conversions

Computers work in **binary (base 2)**, but humans find **denary (base 10)** easier to understand. **Hexadecimal (base 16)** provides a compact way to represent binary values. You need to be able to convert between all three systems.

1.1 Binary to denary conversion

To convert binary to denary, write out the place values (128, 64, 32, 16, 8, 4, 2, 1) and add together the values where there is a 1. This is the most common conversion question in exams and appears almost every year.

Step-by-step method:

1. Write out the 8-bit place values: 128, 64, 32, 16, 8, 4, 2, 1
2. Write the binary number underneath, aligning each bit with its place value
3. For each column where the bit is 1, write down the place value
4. Add all the place values together

Example: Convert 11110000 to denary.

128	64	32	16	8	4	2	1
1	1	1	1	0	0	0	0

Add: 128 + 64 + 32 + 16 = **240**

Practice tip: The 2024 exam provided a table with place values already printed. You just needed to identify which columns had 1s and add them. Make sure you can do this quickly and accurately — it's a straightforward 1-mark question that you shouldn't lose marks on.

1.2 Denary to binary conversion

To convert denary to binary, start with the largest place value (128) and work right. If the place value fits into your remaining number, put a 1 and subtract it. If not, put a 0. This is sometimes called the "subtraction method".

Step-by-step method:

1. Start with your denary number and the largest place value (128)
2. Ask: "Does 128 fit into my number?" If yes, put 1 and subtract 128. If no, put 0.
3. Move to the next place value (64) and repeat
4. Continue until you reach the 1s column
5. Your remaining number should be 0

Example: Convert 105 to 8-bit binary.

- 128? No (128 > 105), put 0. Remaining: 105
- 64? Yes (64 ≤ 105), put 1. Remaining: 105 − 64 = 41
- 32? Yes (32 ≤ 41), put 1. Remaining: 41 − 32 = 9
- 16? No (16 > 9), put 0. Remaining: 9
- 8? Yes (8 ≤ 9), put 1. Remaining: 9 − 8 = 1
- 4? No (4 > 1), put 0. Remaining: 1
- 2? No (2 > 1), put 0. Remaining: 1
- 1? Yes (1 ≤ 1), put 1. Remaining: 0 ✓

Result: **01101001**

Verification: Always check your answer by converting back: 64 + 32 + 8 + 1 = 105 ✓

Alternative method — repeated division: You can also divide the number by 2 repeatedly, recording the remainders. The remainders read upward give the binary number. However, the subtraction method is usually faster for exam questions.

1.3 Number ranges and bit requirements

The number of bits determines how many different values can be represented. This is a crucial concept that appears in multiple contexts: character sets, colour depth, file sizes, and more.

Key formulas:

- **n bits** can represent **2^n different values**
- The **minimum** value (unsigned) is always **0**
- The **maximum** value is **$2^n - 1$**

Why maximum = $2^n - 1$: With n bits, you have 2^n possible combinations. But since we start counting from 0, the highest value is one less than the number of combinations. For example, with 4 bits you have 16 combinations (0000 to 1111), representing values 0 to 15.

Finding minimum bits for a given number of values: Work through powers of 2 until you find one big enough.

- 2^1 = 2 values
- 2^2 = 4 values
- 2^3 = 8 values
- 2^4 = 16 values

- 2^5 = 32 values
- 2^6 = 64 values
- 2^7 = 128 values
- 2^8 = 256 values

Common exam application — character sets: If a character set contains 150 characters, how many bits are needed? You need enough bits to represent 150 different values. 2^7 = 128 (not enough), 2^8 = 256 (enough). Answer: **8 bits**.

Common exam application — colour depth: If you have 7-bit colour depth, you can represent 2^7 = **128 different colours**. This was asked in the 2024 exam.

Common exam application — sound samples: If a sound file uses 16-bit samples, each sample can have 2^{16} = 65,536 different values, allowing for more precise representation of the sound wave.

1.4 Why hexadecimal is useful

Hexadecimal (hex) uses 16 digits: 0—9 and A—F (where A=10, B=11, C=12, D=13, E=14, F=15). Each hex digit represents exactly 4 binary bits, which makes it perfectly suited for computing where data is often handled in 4-bit or 8-bit chunks.

Benefits of hexadecimal:

- More compact than binary — an 8-bit binary number becomes just 2 hex digits
- Easier for humans to read, write, and remember than long binary strings
- Easy to convert to/from binary (each hex digit = 4 bits) — no calculation needed, just lookup
- Less prone to errors when typing or copying — it's easier to spot a mistake in "B7" than in "10110111"
- Still represents binary data accurately — unlike denary, which doesn't map cleanly to binary

Why not just use denary? Denary doesn't map neatly to binary. Converting denary to binary requires division, and there's no simple way to tell what the binary looks like from the denary number. With hex, each digit directly corresponds to 4 bits, so you can mentally "see" the binary.

Common uses of hexadecimal:

- **Memory addresses:** 0x7FFF is clearer than 0111111111111111. Programmers can quickly see which bits are set.
- **Colour codes:** #FF5733 represents RGB values (FF=red, 57=green, 33=blue). Web developers use this format constantly.
- **MAC addresses:** 00:1A:2B:3C:4D:5E identifies network devices. Each pair is one byte.
- **Error codes:** 0x80070005 is easier to search for and remember than a binary or denary equivalent.
- **Machine code:** Assembly language and debugging tools display instructions and data in hex.
- **Unicode characters:** Characters like emojis are often referenced by their hex code point (e.g. U+1F600 for 😀).

1.5 Binary ↔ Hexadecimal conversion

Converting between binary and hexadecimal is straightforward because each hex digit maps exactly to 4 binary bits. This is the key advantage of hexadecimal — no arithmetic required, just pattern matching.

Binary to hex: Split the binary into groups of 4 bits (nibbles), starting from the right. If the leftmost group has fewer than 4 bits, pad it with leading zeros. Then convert each group to its hex digit using the table you memorised.
Example: 11110000 → 1111 | 0000 → F | 0 → **F0**
Another example: 101011 → pad to 8 bits: 00101011 → 0010 | 1011 → 2 | B → **2B**
Hex to binary: Convert each hex digit to its 4-bit binary equivalent. Concatenate the results. This is essentially a lookup operation.
Example: A3 → A=1010, 3=0011 → **10100011**
Key insight: Memorise the binary for each hex digit (especially A-F). This makes conversions almost instant:

- A = 1010 (ten)
- B = 1011 (eleven)
- C = 1100 (twelve)
- D = 1101 (thirteen)
- E = 1110 (fourteen)
- F = 1111 (fifteen — all bits on)

1.6 Denary ↔ Hexadecimal conversion

Converting between denary and hexadecimal requires a bit more arithmetic than binary-hex conversions, but the methods are straightforward once you understand them.
Denary to hex (direct method): Divide by 16. The remainder is the right digit, the quotient is the left digit. If either value is 10-15, convert it to A-F.
Example: Convert 200 to hex.

- 200 ÷ 16 = 12 remainder 8
- 12 = C, 8 = 8
- Result: **C8**

Verification: Check by converting back: 12 × 16 + 8 = 192 + 8 = 200 ✓
Alternative method (via binary): Convert denary to binary first, then split into nibbles and convert to hex. This is more steps but less error-prone for larger numbers.
Hex to denary: The left digit represents how many 16s, the right digit represents how many 1s. Multiply the left digit by 16 and add the right digit.
Example: Convert B7 to denary.

- B = 11, 7 = 7
- 11 × 16 + 7 = 176 + 7 = **183**

Why this works: Just like denary has a tens column and a units column, hex has a sixteens column and a units column. The left digit tells you how many sixteens, the right tells you how many units.

> **Exam tip:** For hex to denary, the 2024 exam asked candidates to "use an example in your answer". Always show the calculation: "B = 11, so B7 = 11 × 16 + 7 = 183".

Worked Example 1 – Binary to denary (from 2024 exam)

Question: Convert the 8-bit binary number 11110000 into denary. [1 mark]

Method:
Write out place values and add where there's a 1:

128 + 64 + 32 + 16 = **240**

Exam tip: This is a quick 1-mark question. Write the answer confidently. You can show brief working in the margin if it helps you.

Worked Example 2 — Denary to binary (from 2024 exam)

Question: Convert the denary number 105 into 8-bit binary. [1 mark]

Method:
Work through place values: 105 = 64 + 32 + 8 + 1

128	64	32	16	8	4	2	1
0	1	1	0	1	0	0	1

Answer: **01101001**

Exam tip: Always give 8 bits, including leading zeros. "1101001" without the leading zero may lose the mark.

Worked Example 3 — Number range calculations (from 2024 exam)

Question: Complete the table by writing the answer to each statement. [4 marks]

The smallest denary number that can be represented by a 4-bit binary number:
Answer: **0** (all bits are 0: 0000)
The largest denary number that can be represented by a 6-bit binary number:
$2^6 - 1 = 64 - 1 =$ **63** (all bits are 1: 111111)
The maximum number of different colours that can be represented with a colour depth of 7-bits:
$2^7 =$ **128** different colours
The minimum number of bits needed to represent 150 different characters in a character set:
$2^7 = 128$ (not enough), $2^8 = 256$ (enough). Answer: **8 bits**

Exam tip: Memorise that the maximum value is $2^n - 1$ and the number of different values is 2^n. These formulas appear frequently.

Worked Example 4 – Hexadecimal to denary (from 2024 exam)

Question: Describe how to convert a 2-digit hexadecimal number into denary. Use an example in your answer. [3 marks]

Model answer:
Convert each hex digit to its denary equivalent (A=10, B=11, etc.) **(1)**.
Multiply the left digit by 16 and add the right digit **(1)**.
Example: B7 → B=11, 7=7 → 11 × 16 + 7 = 176 + 7 = 183 **(1)**.

Exam tip: When asked to "describe how to convert", give the method as clear steps. When asked to "use an example", you must include one to get full marks.

2. Binary arithmetic: addition and shifts

Computers perform calculations in binary. You need to be able to add binary numbers and perform binary shifts, understanding how these operations affect the values.

2.1 Binary addition

Binary addition follows simple rules, similar to denary addition but with only two digits (0 and 1). Just like in denary, when the sum of a column exceeds the largest single digit, you carry to the next column.

The four basic rules:

- **0 + 0 = 0** (nothing plus nothing)
- **0 + 1 = 1** (nothing plus one)
- **1 + 0 = 1** (one plus nothing)
- **1 + 1 = 10** (write 0, carry 1 to the next column — like 5+5=10 in denary)
- **1 + 1 + 1 = 11** (write 1, carry 1 to the next column — this happens when adding a carry)

Work from right to left, just like denary addition. Write carry digits above the next column so you don't forget them.

Column-by-column approach: For each column, add the two bits plus any carry from the previous column. If the result is 0 or 1, write it down. If the result is 2, write 0 and carry 1. If the result is 3, write 1 and carry 1.

2.2 Overflow

Overflow occurs when the result of a calculation is too large to fit in the available number of bits. For example, adding two 8-bit numbers might produce a 9-bit result, but the 9th bit has nowhere to go in an 8-bit system.

When overflow occurs, the extra bit is lost, and the result wraps around to a smaller number. This can cause serious errors in programs if not handled correctly. For example, in early video games, player scores could "wrap around" from maximum to zero.

Example of overflow: 11111111 (255) + 00000001 (1) = 100000000, but only 8 bits are available, so the result is 00000000 (0) — clearly wrong! The carry bit that would make 256 is lost.

Detecting overflow: If adding two positive numbers gives a result smaller than either input, overflow has occurred. The 2024 exam asked candidates to perform binary addition; always check your result makes sense.

Preventing overflow: Programmers use larger data types (16-bit, 32-bit, or 64-bit integers) when working with large numbers, or add checks to detect when overflow might occur.

2.3 Showing working

In exam questions, always show your working by writing carry digits. This ensures you get method marks even if you make a small error.

Format your addition neatly with columns aligned:

```
  1 1 1 1     ← carries
  0 1 1 0 1 0 1 1
+ 0 0 0 0 1 1 1 1
─────────────────
  0 1 1 1 1 0 1 0
```

2.4 Binary shifts

A **binary shift** moves all the bits in a number left or right by a specified number of places. Shifts are a fast way to multiply or divide by powers of 2 — much faster than actual multiplication or division, which is why processors use them for optimisation.

Left shift

Bits move left, and zeros fill in from the right. Each left shift by 1 place **multiplies the value by 2**. Think of it like this: shifting left moves each bit into a column with double the place value.

- Left shift by 1: × 2
- Left shift by 2: × 4 (× 2^2)
- Left shift by 3: × 8 (× 2^3)
- Left shift by n: × 2^n

Warning: Bits that shift off the left edge are lost. If a 1 bit is lost, the result will be incorrect (similar to overflow). For example, left-shifting 10000000 by 1 place gives 00000000 — the leading 1 is lost.

Memory aid: Left shift = Larger (L = L). The number gets bigger.

Right shift

Bits move right, and zeros fill in from the left. Each right shift by 1 place **divides the value by 2** (integer division — any remainder is lost). Think of it like this: shifting right moves each bit into a column with half the place value.

- Right shift by 1: ÷ 2
- Right shift by 2: ÷ 4 (÷ 2^2)
- Right shift by 3: ÷ 8 (÷ 2^3)
- Right shift by n: ÷ 2^n

Note: Bits that shift off the right edge are lost. This means odd numbers lose their remainder when right-shifted (e.g. 7 ÷ 2 = 3, not 3.5). Binary cannot represent fractions in this simple format.

Memory aid: Right shift = Reduce (R = R). The number gets smaller.

Practical application: The 2024 exam asked candidates to perform a left shift of 4 places. This multiplies by 2^4 = 16. If you understand the relationship between shifts and

multiplication/division, you can verify your answer.

Exam tip: Left shift = multiply (think: L for Larger). Right shift = divide (think: R for Reduce). A shift of n places means × 2^n or ÷ 2^n. This pattern is guaranteed to appear in exams.

Worked Example 5 — Binary addition (from 2024 exam)

Question: Add these two 8-bit binary numbers using binary addition. Show your working. [2 marks]

`01101011 + 00001111`

Working:

```
Carries:  1 1 1 1 1 1
          0 1 1 0 1 0 1 1   (107)
        + 0 0 0 0 1 1 1 1   (15)
        ------------------
          0 1 1 1 1 0 1 0   (122)
```

Answer: **01111010**

Exam tip: Write your carries clearly. Column by column from right: 1+1=10 (write 0, carry 1), 1+1+1=11 (write 1, carry 1), and so on.

Worked Example 6 — Left binary shift (from 2024 exam)

Question: Show the result of a left binary shift of 4 places on the binary number 00001111. [1 mark]

Method:
Original: 00001111 (denary: 15)
Shift all bits 4 places left, fill with zeros on the right:
Result: **11110000** (denary: 240 = 15 × 16 = 15 × 2^4)

Exam tip: A left shift of 4 places multiplies by 2^4 = 16. You can verify: 15 × 16 = 240, and 11110000 = 128+64+32+16 = 240. ✓

Worked Example 7 — Right binary shift

Question: Perform a right shift of 2 places on the binary number 01100100. State the denary value before and after the shift. [2 marks]

Method:
Original: 01100100 (denary: 64 + 32 + 4 = 100)
Shift all bits 2 places right, fill with zeros on the left:
Result: **00011001** (denary: 16 + 8 + 1 = 25)
Check: 100 ÷ 4 = 25 ✓ (right shift by 2 = ÷ 2^2 = ÷ 4)

Exam tip: When verifying, remember that right shift performs integer division — any remainder is discarded.

Worked Example 8 – Explaining the effect of a shift

Question: Explain what happens to the denary value when a binary number is left-shifted by 3 places. [2 marks]

Model answer:
The denary value is multiplied by 8 **(1)** because each left shift multiplies by 2, so 3 shifts multiplies by 2 × 2 × 2 = 2^3 = 8 **(1)**.

Exam tip: Always state the effect (multiplied/divided) AND the amount (2^n). Explaining why (each shift = ×2 or ÷2) can earn extra marks.

3. Hexadecimal in depth

Hexadecimal is widely used in computing because it provides a compact, human-readable representation of binary data. This section covers more complex conversions and practical applications.

3.1 Why hexadecimal exists

While computers work in binary, humans find long binary strings hard to read and prone to errors. Hexadecimal solves this problem because:

- Each hex digit represents exactly 4 binary bits
- 8 binary digits become just 2 hex digits
- It's much easier to spot errors in "FF" than in "11111111"
- Conversion between hex and binary is straightforward

Programmers use hex for:

- **Memory addresses:** 0x7FFF is clearer than 0111111111111111
- **Colour codes:** #FF5733 represents RGB values
- **MAC addresses:** 00:1A:2B:3C:4D:5E
- **Error codes:** 0x80070005 is easier to search for than a binary equivalent
- **Machine code:** Assembly language uses hex for opcodes and addresses

3.2 Multi-step conversions

Sometimes you need to convert via an intermediate format:

Denary → Hex (via binary):

1. Convert denary to binary
2. Split binary into nibbles (4-bit groups)
3. Convert each nibble to hex

Hex → Denary (via binary):

1. Convert each hex digit to 4-bit binary
2. Combine to get full binary number
3. Convert binary to denary

Direct method is often faster for 2-digit hex, but the binary method works for any size and is less error-prone.

3.3 Colour codes example

Web colours use 6-digit hex codes in the format #RRGGBB, where each pair represents the intensity of Red, Green, and Blue (0—255 in denary, 00—FF in hex).

Example: #FF5733

- Red: FF = 255 (maximum)
- Green: 57 = 5×16 + 7 = 87
- Blue: 33 = 3×16 + 3 = 51

This produces an orange-red colour with lots of red, some green, and a little blue.

3.4 Common hex values to memorise

Hex	Denary	Binary	Use
00	0	00000000	Minimum value
0F	15	00001111	Max 4-bit value
10	16	00010000	First 2-digit hex
64	100	01100100	Denary 100
7F	127	01111111	Max signed 8-bit
80	128	10000000	2^7
FF	255	11111111	Max 8-bit value

Exam tip: Memorising FF = 255 and 00 = 0 helps you quickly check your conversions. If your answer is outside 00—FF for an 8-bit number, something went wrong.

Worked Example 9 — Binary to hexadecimal

Question: Convert the binary number 10110111 to hexadecimal. [2 marks]

Method:
Step 1: Split into nibbles (4-bit groups from right): 1011 | 0111
Step 2: Convert each nibble:

- 1011 = 8+2+1 = 11 = B
- 0111 = 4+2+1 = 7 = 7

Answer: **B7**

Exam tip: Always split into groups of 4 bits from the right. If you have fewer than 4 bits on the left, pad with zeros (e.g. 110 becomes 0110).

Worked Example 10 — Hexadecimal to binary

Question: Convert the hexadecimal number 2A to 8-bit binary. [2 marks]

Method:
Step 1: Convert each hex digit to 4-bit binary:

- 2 = 0010
- A = 10 = 1010

Step 2: Combine: 0010 1010
Answer: **00101010**

Exam tip: Remember A=10=1010, B=11=1011, C=12=1100, D=13=1101, E=14=1110, F=15=1111. These come up frequently.

Worked Example 11 — Denary to hexadecimal

Question: Convert the denary number 200 to hexadecimal. [2 marks]

Method (direct):
Step 1: Divide by 16: 200 ÷ 16 = 12 remainder 8
Step 2: Convert digits: 12 = C, 8 = 8
Answer: **C8**
Check: 12 × 16 + 8 = 192 + 8 = 200 ✓

Exam tip: The quotient is the left digit, the remainder is the right digit. Always check by converting back.

Worked Example 12 — Explaining why hex is used

Question: Explain two reasons why programmers use hexadecimal instead of binary. [4 marks]

Model answer:
1. Hexadecimal is more compact than binary **(1)**. An 8-bit binary number can be represented with just 2 hex digits, making it easier to read and write **(1)**.
2. Hexadecimal is less error-prone **(1)**. It's easier to spot a mistake in "B7" than in "10110111", reducing errors when entering or copying data **(1)**.

Exam tip: Give two distinct reasons with explanation. "Shorter" and "easier to read" could be seen as the same point. "Shorter" and "less error-prone" are clearly different.

Worked examples summary

The 12 worked examples above cover the main question types you will encounter on data representation. Here is a summary of the key question patterns:

Example	Question type	Key skill
1	Binary → denary (1 mark)	Add place values where there's a 1
2	Denary → binary (1 mark)	Subtract largest place values, include leading zeros
3	Number ranges (4 marks)	2^n values, max = 2^n-1, min bits for n values
4	Hex → denary with method (3 marks)	Describe steps AND give example
5	Binary addition (2 marks)	Show carries clearly
6	Left shift (1 mark)	Move bits left, fill with zeros
7	Right shift with verification	Move bits right, verify ÷ 2^n
8	Explain shift effect (2 marks)	State multiplication/division factor
9	Binary → hex (2 marks)	Split into nibbles, convert each
10	Hex → binary (2 marks)	Convert each digit to 4 bits
11	Denary → hex (2 marks)	Divide by 16, quotient + remainder
12	Explain hex usage (4 marks)	Give two distinct advantages with explanation

Quick reference

Key terms — exam-ready definitions

Term	Definition (mark scheme style)
Binary	Base 2 number system using digits 0 and 1; used by computers because circuits have two states (on/off)
Denary	Base 10 number system using digits 0-9; the everyday number system humans use
Hexadecimal	Base 16 number system using digits 0-9 and A-F; compact way to represent binary
Bit	Binary digit; the smallest unit of data (0 or 1)
Nibble	4 bits; represents one hexadecimal digit
Byte	8 bits; can represent values 0-255
Overflow	When the result of a calculation is too large to fit in the available bits
Binary shift	Moving all bits left or right; left shift multiplies by 2, right shift divides by 2
Place value	The value each column position represents (in binary: 1, 2, 4, 8, 16, 32, 64, 128...)
Kilobyte (KB)	1,000 bytes; used for small files like text documents
Megabyte (MB)	1,000 KB (1,000,000 bytes); used for photos, songs
Gigabyte (GB)	1,000 MB; used for videos, games, applications
Terabyte (TB)	1,000 GB; used for hard drive capacity
Petabyte (PB)	1,000 TB; used for data centre storage

Conversion quick reference

Conversion	Method
Binary → Denary	Add place values where bit = 1
Denary → Binary	Subtract largest place value that fits, put 1; repeat
Binary → Hex	Split into 4-bit groups, convert each to hex digit
Hex → Binary	Convert each hex digit to 4-bit binary
Denary → Hex	Divide by 16: quotient = left digit, remainder = right digit

Conversion	Method
Hex → Denary	Left digit × 16 + right digit

Powers of 2

2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8
1	2	4	8	16	32	64	128	256

Binary addition rules

A	B	Carry In	Sum	Carry Out
0	0	0	0	0
0	1	0	1	0
1	0	0	1	0
1	1	0	0	1
0	0	1	1	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	1

Typical pitfalls

Pitfall 1 — Forgetting leading zeros.
Wrong: 105 in binary is "1101001"
Right: 105 in 8-bit binary is "**0**1101001". Always include leading zeros to make 8 bits.

Pitfall 2 — Confusing max value with number of values.
Wrong: "4 bits can represent up to 16"
Right: 4 bits can represent **16 different values** (0-15). The **maximum value** is 15 (= $2^4 - 1$).

Pitfall 3 — Forgetting to show carries in addition.
Wrong: Just writing the final answer.
Right: Show carry digits above each column. This earns method marks even if the answer is wrong.

Pitfall 4 — Shifting the wrong direction.
Wrong: "Left shift divides by 2"
Right: Left shift **multiplies** (bits move Left = Larger). Right shift **divides** (bits move Right = Reduced).

Pitfall 5 — Not using examples when asked.
Wrong: "Multiply the first digit by 16 and add the second"
Right: Include an example: "B7 = 11×16 + 7 = 183" when the question says "use an example".

Pitfall 6 — Hex letter confusion.
Wrong: Thinking A=1, B=2, etc.
Right: A=10, B=11, C=12, D=13, E=14, F=15. The letters **continue** from 9.

Pitfall 7 — Wrong nibble grouping.
Wrong: Grouping bits from the left: 1011 | 011 → B3
Right: Group from the right: 101 | 1011 → pad left → 0101 | 1011 → 5B

Pitfall 8 — Mixing up quotient and remainder.
Wrong: 200 ÷ 16 = 8 remainder 12, so answer is 8C
Right: 200 ÷ 16 = **12** remainder **8**. Quotient (12=C) is left, remainder (8) is right: **C8**.

Pitfall 9 — Ignoring overflow.
Wrong: "11111111 + 00000001 = 100000000"
Right: In 8-bit systems, overflow occurs. The 9th bit is lost, giving 00000000. This is an error condition.

Pitfall 10 — Vague explanations for hex usage.
Wrong: "Hex is easier to use"
Right: "Hex is more **compact** (8 binary digits = 2 hex digits)" or "Hex is **less error-prone** (easier to spot mistakes in 'B7' than '10110111')."

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.